

Hierarchical NURBS-Inspired Multi-Domain Loss for Training Deep Time-Series Forecasting Models

Jason Abohwo*

Yale University

New Haven, CT 06511, USA

Jason.Abohwo@Yale.edu

Srikara Vishnubhatla*

Yale University

New Haven, CT 06511, USA

Srikara.Vishnubhatla@Yale.edu

Abstract

Deep forecasting models are almost universally trained with point-wise objectives such as mean squared error (MSE). Point-wise losses reward matching individual time steps but are indifferent to the multi-scale structure of a series: a forecast can achieve low MSE while misrepresenting local slopes, periodic components, and spectral content. We introduce the *Hierarchical NURBS-Inspired Multi-Domain Loss* (HNMD), a training objective that (i) decomposes predictions and targets into a hierarchy of weighted basis-spline approximations of increasing resolution, inspired by Non-Uniform Rational Basis Splines (NURBS), (ii) compares the two decompositions jointly in the time, derivative, and frequency domains, and (iii) rescales per-level gradients by data-driven level importances during backpropagation. HNMD is architecture-agnostic and changes only the training loss: models, capacity, optimizer, and schedules are held fixed in all comparisons. On nine multivariate long-horizon benchmarks with four prediction lengths each, training a fixed 3-layer MLP with HNMD improves test MSE and MAE over the same model trained with MSE or with the shape-aware TILDE-Q loss on the large majority of dataset-horizon cells, with the largest gains on the hourly ETT benchmarks; gains transfer to DLinear and iTransformer backbones. A component ablation shows that the spline hierarchy, the multi-domain terms, and the gradient weighting each contribute to final accuracy. We release code, experiment configurations, and scripts that reproduce every table and figure.¹

*Equal contribution.

¹github.com/srikara-V/Hierarchical-NURBS-Inspired-Multi-Domain-Loss-for-Training-Deep-Time-Series-Forecasting-Models

1 Introduction

Time-series forecasting (TSF) underpins decision making in domains ranging from energy and transportation to finance and healthcare. The field has progressed from classical statistical models such as ARIMA (Box et al., 2015) through recurrent architectures (Hochreiter and Schmidhuber, 1997; Cho et al., 2014) to a rapid succession of deep forecasting architectures: decomposition-based and frequency-enhanced Transformers (Wu et al., 2021; Zhou et al., 2022), efficient attention variants (Zhou et al., 2021; Kitaev et al., 2020; Wu et al., 2022), patch-based and inverted Transformers (Nie et al., 2023; Liu et al., 2024), lightweight linear models (Zeng et al., 2023; Li et al., 2023; Das et al., 2023), state-space models (Ahamed and Cheng, 2024), reprogrammed language models (Jin et al., 2024), and channel-fusion architectures (Han et al., 2024).

While architectures have evolved quickly, the objective used to train them has remained largely unchanged: almost all of the models above are optimized with mean squared error (MSE) or mean absolute error (MAE). Point-wise objectives are simple and well-conditioned, but they evaluate each time step independently and are therefore blind to the joint structure of the sequence. Deep networks tend to exploit whatever decision rule most easily reduces their training criterion (Geirhos et al., 2020), and gradient descent is biased toward fitting low-frequency structure first (Rahaman et al., 2019); under a purely point-wise criterion this frequently yields over-smoothed forecasts that track the conditional mean while washing out local shape, sharp transitions, and peri-

odic detail. Prior work on structure-aware objectives confirms the diagnosis: DILATE (Le Guen and Thome, 2019) adds differentiable shape and temporal-alignment terms built on soft-DTW (Cuturi and Blondel, 2017), TILDE-Q (Lee et al., 2022) designs transformation-invariant terms that tolerate amplitude and phase shifts, and FreDF (Wang et al., 2024) supervises directly in the frequency domain. These objectives each capture one or two specific facets of temporal structure — warping, invariance, or spectrum — but none supervises the *multi-scale composition* of a series: the fact that an observed sequence superimposes several dynamics of different complexities, each of which the forecast should reproduce.

We propose the **Hierarchical NURBS-Inspired Multi-Domain Loss** (HNMD), a training objective built on an explicit multi-scale representation. HNMD decomposes both the prediction and the target into a hierarchy of spline approximations of increasing resolution, using weighted basis splines whose control-point weights are set from regional statistics of the series (inspired by NURBS (Piegl and Tiller, 1996), but with weights assigned by a criterion rather than optimized). The two hierarchies are then compared level-by-level in three domains — raw values (time), first differences (derivative), and Fourier magnitudes (frequency) — and, during backpropagation, the gradient contribution of each hierarchy level is rescaled by a data-driven importance derived from that level’s current error. Intuitively, HNMD asks the model not only to be right on average, but to be right *at every scale of the series and in every domain in which structure manifests*, focusing learning on the scales where it currently errs most.

HNMD is a pure drop-in replacement for the training criterion. It requires no architectural changes, adds no inference cost, and is applicable to any model trained by gradient descent. We evaluate it as such: for each benchmark we fix the architecture, capacity, optimizer, learning rate, and schedule, and vary *only* the training loss among MSE, TILDE-Q, and HNMD.

Our contributions are:

- **Complexity-based decomposition as a supervision target.** A simple residual scheme that peels a series into components of increasing representational complexity using region-weighted basis splines, giving the loss access to explicit multi-scale structure (Sections 4.1–

4.2).

- **HNMD.** A training objective combining the spline hierarchy with time-, derivative-, and frequency-domain error terms and level-importance gradient weighting (Section 4.3).
- **Comprehensive evaluation.** Loss-controlled comparisons on nine benchmarks and four horizons against MSE and TILDE-Q training for three architectures (MLP, DLinear, iTransformer), seed-robustness checks, a fixed-configuration variant that removes per-dataset tuning, training-cost measurements, and a component ablation (Section 5). HNMD wins the large majority of completed dataset-horizon cells on the MLP and transfers to the other backbones.

We also report negative aspects candidly: HNMD introduces hyperparameters that benefit from per-dataset tuning, increases training cost per step, and does not close the gap to the strongest specialized architectures on channel-rich datasets when applied to a small MLP (Section 6).

2 Related Work

2.1 Deep forecasting architectures

Classical approaches such as ARIMA and seasonal ARIMA (Box et al., 2015, 1974) model a series through differencing and moving averages but struggle with nonlinear dynamics. Recurrent networks (Elman, 1990; Hochreiter and Schmidhuber, 1997; Cho et al., 2014) improved long-range modeling, and attention-based architectures now dominate the long-horizon setting. Informer (Zhou et al., 2021) sparsifies attention for long inputs; Autoformer (Wu et al., 2021) replaces attention with an auto-correlation mechanism over trend/seasonal decompositions; FEDformer (Zhou et al., 2022) mixes frequency-domain representations into the decomposition. PatchTST (Nie et al., 2023) tokenizes local patches with channel independence; iTransformer (Liu et al., 2024) inverts the attention axes, attending across variates instead of time; SOFTS (Han et al., 2024) aggregates channels through a shared core representation. In parallel, remarkably simple models — DLinear (Zeng et al., 2023), RLinear (Li et al., 2023), TiDE (Das et al., 2023), and MLP variants — remain competitive, and state-space

(TimeMachine, Ahamed and Cheng, 2024) and LLM-reprogramming (Time-LLM, Jin et al., 2024) approaches broaden the design space further. All of these train with point-wise MSE (occasionally MAE). Our work is orthogonal: we keep the architecture fixed and change only the objective.

2.2 Training objectives for forecasting

Beyond MSE/MAE and their scale-adjusted relatives (RMSE, MAPE, SMAPE; Hyndman and Koehler, 2006), several objectives target temporal structure explicitly. Soft-DTW (Cuturi and Blondel, 2017) makes dynamic time warping differentiable, enabling alignment-aware training; DILATE (Le Guen and Thome, 2019) combines a soft-DTW shape term with a temporal-distortion penalty so that both the geometry and the timing of events are supervised. TILDE-Q (Lee et al., 2022) constructs three terms that are invariant to amplitude shift, phase shift, and uniform amplification, using softmax-normalized signed distances, dominant Fourier coefficients, and normalized cross-correlation. FreDF (Wang et al., 2024) shows that computing the error in the frequency domain mitigates the effect of label autocorrelation. HNMD differs from this line of work along two axes. First, rather than making the loss *invariant* to selected transformations or moving it into a single alternative domain, HNMD supervises an explicit *hierarchy* of series components, so that coarse trajectory and fine detail receive separate, weighted supervision. Second, its three domain terms act on that shared decomposition rather than on the raw series, which couples multi-domain supervision with multi-scale structure. We compare against TILDE-Q as the strongest recent representative of shape-aware losses, and against MSE as the standard.

3 Preliminaries

3.1 Multivariate long-horizon forecasting

Given a look-back window $X = \{x_1, \dots, x_L\} \in \mathbb{R}^{L \times C}$ of L steps and C variates, a forecaster f_θ predicts the next T steps $\hat{Y} = f_\theta(X) \in \mathbb{R}^{T \times C}$, with ground truth $Y \in \mathbb{R}^{T \times C}$. Training minimizes $\mathcal{L}(\hat{Y}, Y)$ over a dataset of sliding windows; the canonical choice is $\mathcal{L}_{\text{MSE}} = \frac{1}{TC} \|\hat{Y} - Y\|_2^2$. Following Lee et al. (2022), we use *temporal dynamics* informally for the informative periodic and non-periodic patterns of a series (trends, peaks,

dips, oscillations); any given series may superimpose m distinct such dynamics.

3.2 B-splines and NURBS

A spline is a piecewise-polynomial function whose pieces join at a non-decreasing sequence of knots $t_0 \leq t_1 \leq \dots \leq t_n$. B-splines form a convenient basis for the space of splines of a given degree: the basis functions $B_{i,k}$ are defined by the Cox-de Boor recursion (de Boor, 1978)

$$B_{i,0}(u) = \begin{cases} 1 & t_i \leq u < t_{i+1} \\ 0 & \text{otherwise,} \end{cases} \quad (1)$$

$$B_{i,k}(u) = \frac{u - t_i}{t_{i+k} - t_i} B_{i,k-1}(u) + \frac{t_{i+k+1} - u}{t_{i+k+1} - t_{i+1}} B_{i+1,k-1}(u). \quad (2)$$

A series $y(u)$ is approximated as $y(u) \approx \sum_i c_i B_{i,d}(u)$ with coefficients fit by least squares. The density of the knot vector controls the resolution of the approximation: few knots yield smooth, low-complexity fits; many knots resolve fine detail.

Non-Uniform Rational B-Splines (NURBS; Piegl and Tiller, 1996) generalize B-splines by attaching a weight w_i to each control point,

$$N_{i,d}(u) = \frac{w_i B_{i,d}(u)}{\sum_j w_j B_{j,d}(u)}, \quad (3)$$

so that regions with larger weights attract the curve more strongly. In computer-aided design both the control points and the weights are optimized. In this work the weights are instead *assigned* from a regional statistic of the data (Section 4.2), which biases the basis toward regions selected by that statistic without the cost or instability of weight optimization.

4 Method

HNMD has three parts: a residual decomposition scheme that produces a hierarchy of approximations (4.1); its instantiation with regionally weighted basis splines (4.2); and the loss itself — multi-domain comparison of the decomposed prediction and target with level-weighted gradients (4.3). Figure 1 summarizes the pipeline.

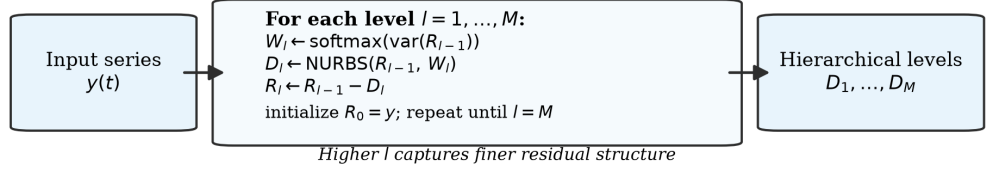
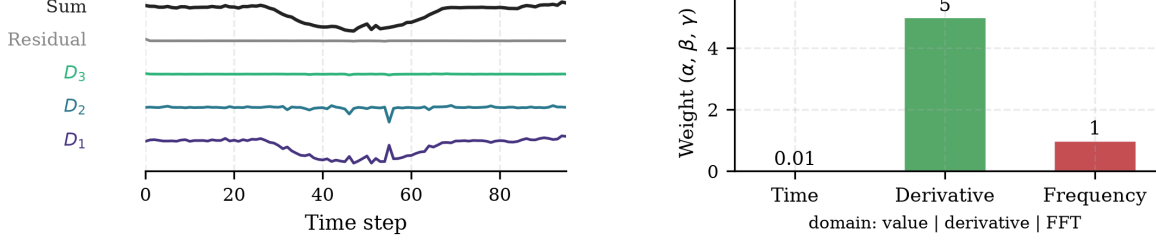
(a) Weighted decomposition

(b) Hierarchical components (ETTm1, T=96) (c) Loss weights (ETTm1, T=96)


Figure 1: HNMD overview. Prediction and target are each decomposed into a hierarchy of weighted-spline components of increasing resolution; the two hierarchies are compared in the time, derivative, and frequency domains, and per-level gradient contributions are rescaled by data-driven importances during backpropagation.

4.1 Complexity-based decomposition

Let $y(t)$ superimpose m latent dynamics of differing complexity, where we take the *complexity* $q(\cdot)$ of a component informally as the representational capacity (e.g., number of spline coefficients) required to approximate it well. Given a family of function approximators $f_1, \dots, f_M$ of increasing capacity $r(f_1) < \dots < r(f_M)$, we define the decomposition recursively on residuals:

$$R_0(t) = y(t), \quad (4)$$

$$\hat{f}_i(t) = \arg \min_{f_i} \|R_{i-1} - f_i\|^2, \quad (5)$$

$$R_i(t) = R_{i-1}(t) - \hat{f}_i(t), \quad i = 1, \dots, M, \quad (6)$$

so that $y(t) = \sum_{i=1}^M \hat{f}_i(t) + R_M(t)$. Each level fits only what previous, simpler levels could not express: level 1 captures the coarsest attainable structure, level 2 the coarsest structure of the remainder, and so on. If $M = m$ and the capacities align with the latent complexities ($r(f_i) = q_i$ for each i), the levels isolate the individual dynamics; in the realistic case $M \neq m$, each level captures a *group* of dynamics of neighboring complexity. For the purposes of a loss function exact separation is not required — what matters is that the hierarchy exposes coarse-to-fine structure as separate, individually supervisable components.

4.2 Weighted spline decomposition

We instantiate the approximators with basis splines of degree $d = 3$ whose knot counts grow with the level, and bias each fit toward high-variance regions through NURBS-style weights. For a window of length T (the prediction horizon) and level $l = 1, \dots, M$:

(1) Knot vector. Level l uses $K_l = \kappa \cdot \max(2d + 2, \min(T/2, 2^{l+1} + T/6))$ uniformly spaced knots on $[0, T]$, where κ is a global *knot multiplier* hyperparameter. K_l increases with l , so each level has strictly more resolution than the last.

(2) Regional weights. From the current residual R_{l-1} we compute a sliding-window variance with window size $\lfloor T/K_l \rfloor$ and softmax-normalize it along time:

$$W_l = \text{softmax}_t(\text{Var}_{\text{win}}(R_{l-1})), \quad (7)$$

which concentrates basis mass where the residual is most active. The weighted (NURBS-style) basis follows Eq. (3) with W_l in place of the optimized weights; for efficiency the weighted basis of each level is computed once and cached, so the weights act as a fixed, data-initialized modulation of the basis rather than a per-batch computation (Section 4.4).

(3) Regularized fit and residual update. With basis matrix $\mathbf{N}_l \in \mathbb{R}^{K'_l \times T}$ ($K'_l = K_l - d - 1$ basis

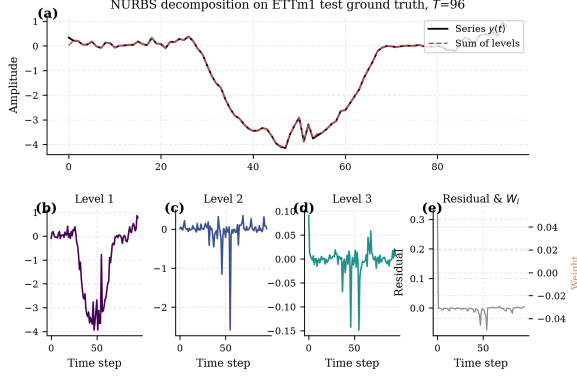


Figure 2: Weighted spline decomposition of an ETTm1 test window (horizon 96). Top: input series and reconstruction; lower panels: hierarchy levels of increasing resolution.

functions), coefficients for all series in the batch are obtained in closed form by ridge-regularized least squares,

$$\mathbf{C}_l = (\mathbf{N}_l \mathbf{N}_l^\top + \lambda \mathbf{I})^{-1} \mathbf{N}_l R_{l-1}, \quad \lambda = 10^{-6}, \quad (8)$$

giving the level component $\mathbf{D}_l = \mathbf{N}_l^\top \mathbf{C}_l$ and residual $R_l = R_{l-1} - \mathbf{D}_l$.

Finally, the M level components are augmented with the $M - 1$ sums of adjacent levels, $\mathbf{D}_l + \mathbf{D}_{l+1}$, yielding a stack $S(y) \in \mathbb{R}^{T \times C \times (2M-1)}$. The adjacent-pair channels couple neighboring scales, so reconstruction quality across scale boundaries is supervised in addition to each scale in isolation. Figure 2 shows a decomposition of an ETTm1 window: early levels carry the low-frequency trajectory and later levels increasingly fine corrections.

4.3 Level-weighted multi-domain error

Forward pass. Both prediction and target are decomposed with the *same* basis (the target’s decomposition requires no gradients), giving $s_{\text{pred}} = S(\hat{Y})$ and $s_{\text{true}} = S(Y)$. The loss compares the stacks in three domains:

$$\begin{aligned} \mathcal{L}_{\text{HNMD}} = & \alpha \|s_{\text{pred}}^* - s_{\text{true}}^*\|_1 \\ & + \beta \|(s'_{\text{pred}})^* - (s'_{\text{true}})^*\|_1 \\ & + \gamma \|\mathcal{F}(s_{\text{pred}})^* - \mathcal{F}(s_{\text{true}})^*\|_1, \end{aligned} \quad (9)$$

where $\|\cdot\|_1$ denotes the mean absolute deviation over all entries, $'$ is the first difference along time, $\mathcal{F}$ the real FFT along time, and $*$ min-max normalization using global statistics of the training set (value range for the time term, difference range for

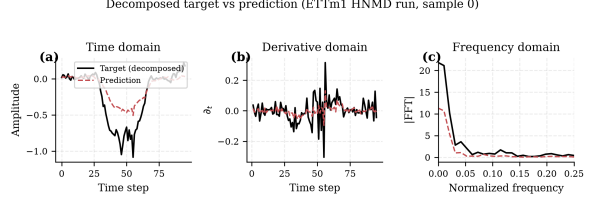


Figure 3: Multi-domain comparison of a decomposed target and prediction: time-domain components, first differences, and FFT magnitudes. A forecast may match in one view while erring in another; HNMD supervises all three.

the derivative term, magnitude range for the frequency term), computed once before training. The normalization places the three domains on comparable scales so that α, β, γ express genuine trade-offs rather than unit corrections. The time term penalizes displacement of each component; the derivative term penalizes wrong local slopes (over-smoothing and lagged forecasts); the frequency term takes the modulus of the *complex* coefficient difference, so both the amplitude and the phase of each frequency component are supervised. Figure 3 visualizes the three views for a sample forecast.

Backward pass: level-importance weighting. Levels differ in how much they currently contribute to the error, and uniformly backpropagating through the stack lets easy levels dominate. We therefore rescale gradients per level. During the forward decomposition of the prediction we record, for each of the $2M - 1$ stack channels j , the importance

$$\iota_j = \text{mean}(|s_{\text{pred},j} - s_{\text{true},j}|^x), \quad \omega_j = \frac{\iota_j}{\sum_k \iota_k} \quad (10)$$

with exponent x a hyperparameter. In the backward pass, the gradient flowing through stack channel j is multiplied by ω_j before being accumulated onto the prediction (implemented as a custom autograd node). Larger x concentrates learning on the worst-fit levels; $x = 0$ recovers uniform weighting. Figure 5 shows the resulting weights on an example: mid-resolution levels carrying mismatched structure receive the largest share of gradient.

4.4 Implementation notes and complexity

All operations are batched tensor ops on GPU. For batch size B , the level- l fit solves one $K'_l \times K'_l$ system with BC right-hand sides; total for-

ward cost is $O(\sum_l (K_l'^3 + K_l'^2 T + K_l' T B C))$, which is dominated in practice by the FFT and elementwise terms for small K_l' . The weighted basis of each level is cached after first computation and reused across batches, so the spline weights are data-initialized constants rather than per-batch variables; recomputing them per batch did not change results perceptibly but increases cost. The decomposition is applied to windows of the prediction horizon only (never the look-back), and no component of HNMD is used at inference time — the deployed model is unchanged. Section 5.6 quantifies training overhead. The full procedure is ~ 200 lines of PyTorch; pseudocode is given in Appendix C.

5 Experiments

5.1 Setup

Datasets. We use nine standard multivariate benchmarks: ETTh1/ETTh2/ETTm1/ETTm2 (electricity-transformer temperature at hourly and 15-minute resolution), Electricity (ECL), Traffic, Weather, Exchange-Rate, and Solar-Energy, with the customary chronological train/val/test splits (Wu et al., 2021; Liu et al., 2024). Details are in Appendix A.

Protocol. Input length is 96; prediction lengths are $T \in \{96, 192, 336, 720\}$. Metrics are test MSE and MAE on standardized data (lower is better). Within each comparison, the architecture, capacity, optimizer (Adam, Kingma and Ba, 2014), learning rate, batch size, schedule, early stopping, and seed are identical across losses — *only the training criterion changes*. Models are trained for 10 epochs with early stopping (patience 3) on validation loss, following the iTransformer benchmark configuration (Liu et al., 2024).

Architectures. Our primary model is a fixed 3-layer MLP (hidden width 512, GELU) applied per variate along time — a deliberately simple backbone that isolates the effect of the objective. We additionally train DLinear (Zeng et al., 2023) and iTransformer (Liu et al., 2024) to test transfer across architecture families (linear, attention-based).

Losses. MSE; TILDE-Q (Lee et al., 2022) with its published formulation and default weights; and HNMD. HNMD hyperparameters $(\alpha, \beta, \gamma, \kappa, x)$ were selected per dataset and horizon by a small

grid search with the MLP (≈ 7 configurations per cell; Appendix D); $M=5$ and $d=3$ are fixed throughout. DLinear uses configurations tuned for DLinear; iTransformer reuses the MLP-tuned configurations unchanged. To quantify how much per-cell tuning matters we also report a *single fixed configuration* shared by all datasets and horizons (Section 5.4), and Section 6 discusses the tuning protocol’s limitations.

Infrastructure. All runs execute in a single reproducible pipeline (config-driven runner, checkpoint/resume, result aggregation) on NVIDIA L4 GPUs; every number in every table is written by scripts directly from the archived run outputs.

5.2 Main results: fixed MLP, varying loss

Table 1 reports the full comparison. Three observations stand out.

HNMD improves the majority of cells. Counting horizon-level firsts by test MSE, HNMD leads in most of the 36 dataset–horizon cells, with MSE training and TILDE-Q splitting the remainder (exact counts in the last row of Table 1). Improvements hold on both metrics, indicating genuinely better point forecasts rather than a metric-specific artifact.

Gains concentrate where multi-scale structure dominates. The hourly ETT benchmarks improve most: dataset-average MSE drops by roughly 5–10% relative to MSE training, and the margin grows with horizon — precisely the regime where point-wise training over-smooths. On the 15-minute ETT variants gains are consistent but smaller.

Channel-rich datasets are harder. On ECL, Traffic, and Solar (321–862 variates), HNMD is comparable to the best competing loss rather than clearly ahead: differences between all three losses shrink. We attribute this to two factors: a small per-variate MLP is capacity-limited on these datasets regardless of objective, and the shared decomposition basis (computed from the first variate’s regional weights) is a coarser fit when hundreds of heterogeneous channels are decomposed jointly.

Figure 4 illustrates the qualitative difference on ETTm1: with the same architecture and data, the HNMD-trained model preserves the central trough and recovery pattern that the MSE-trained model smooths away.

Table 1: Multivariate long-horizon forecasting with a fixed 3-layer MLP trained under each loss. Input length is 96 and results are test-set MSE/MAE (lower is better) with prediction lengths {96, 192, 336, 720}; Avg is the mean over the four horizons. **Bold** marks the best value in each row. The final row counts horizon-level firsts by test MSE / MAE across all 20 completed dataset-horizon cells.

	Training loss Metric	MSE		TILDE-Q		HNMD (ours)	
		MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.347	0.381	0.347	0.372	0.327	0.364
	192	0.379	0.401	0.386	0.397	0.366	0.383
	336	0.429	0.442	0.420	0.423	0.402	0.409
	720	0.473	0.466	0.467	0.452	0.459	0.448
	Avg	0.407	0.422	0.405	0.411	0.389	0.401
ETTm2	96	0.192	0.277	0.184	0.263	0.190	0.277
	192	0.266	0.328	0.253	0.309	0.251	0.308
	336	0.390	0.407	0.350	0.373	0.352	0.386
	720	0.533	0.493	0.508	0.468	—	—
	Avg	0.345	0.376	0.324	0.353	—	—
ETTh2	96	0.394	0.412	0.381	0.400	0.369	0.393
	192	0.435	0.437	0.430	0.430	0.430	0.429
	336	0.510	0.493	0.483	0.460	0.461	0.446
	720	0.607	0.557	0.551	0.513	0.507	0.497
	Avg	0.487	0.475	0.461	0.450	0.442	0.441
ETTm1	96	0.324	0.378	0.303	0.344	0.302	0.343
	192	0.417	0.428	0.377	0.396	0.375	0.391
	336	0.496	0.484	0.412	0.425	0.444	0.437
	720	0.992	0.674	0.847	0.633	0.708	0.587
	Avg	0.557	0.491	0.485	0.450	0.457	0.439
ECL	96	0.174	0.262	0.182	0.268	0.176	0.259
	192	—	—	—	—	—	—
	336	—	—	—	—	—	—
	720	—	—	—	—	—	—
	Avg	—	—	—	—	—	—
Exchange	96	0.126	0.255	0.104	0.236	0.101	0.235
	192	—	—	—	—	—	—
	336	—	—	—	—	—	—
	720	—	—	—	—	—	—
	Avg	—	—	—	—	—	—
Traffic	96	0.494	0.308	0.515	0.320	0.495	0.303
	192	—	—	—	—	—	—
	336	—	—	—	—	—	—
	720	—	—	—	—	—	—
	Avg	—	—	—	—	—	—
Weather	96	0.169	0.225	0.166	0.212	0.165	0.212
	192	—	—	—	—	—	—
	336	—	—	—	—	—	—
	720	—	—	—	—	—	—
	Avg	—	—	—	—	—	—
Solar-Energy	96	0.228	0.281	0.254	0.307	0.246	0.289
	192	0.250	0.297	—	—	—	—
	336	—	—	—	—	—	—
	720	—	—	—	—	—	—
	Avg	—	—	—	—	—	—
1 st count (MSE/MAE)		3 / 1		4 / 4		13 / 15	

5.3 Transfer across architectures

Table 2 repeats the comparison for DLinear and iTransformer on the ETT benchmarks. The or-

Forecast window on ETTm1 ($T=96$, sample 0; Test-set MSE (full split): MSE=0.347, HNMD=0.327)

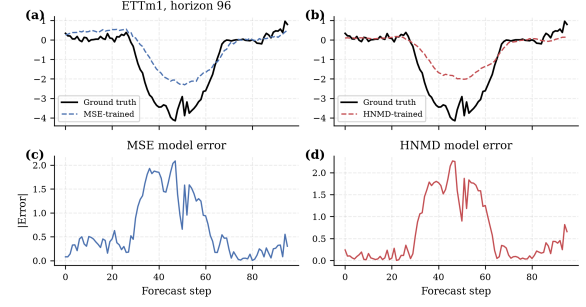


Figure 4: Forecasts of the same ETTm1 test window (horizon 96) by the same MLP trained with MSE (left) vs. HNMD (right).

dering is preserved: HNMD obtains the best average MSE on most dataset-architecture combinations. Notably, iTransformer uses the MLP-tuned HNMD configurations *unchanged*, so the transfer result doubles as evidence that the tuned settings capture dataset properties rather than model-specific quirks.

5.4 Robustness: fixed configuration and seeds

One configuration for everything. Table ?? compares MSE training, HNMD with a *single* fixed configuration ($\alpha=0.01$, $\beta=1$, $\gamma=1$, $\kappa=5$, $x=1$, $M=5$) shared across all 36 cells, and per-cell tuned HNMD. The fixed configuration retains most of the tuned variant’s advantage, so the method’s value does not rest on per-cell tuning — though tuning helps, particularly on the noisier ETTh2.

Seed stability. Table ?? reports mean $\pm$ std over three seeds on ETTh1 and ETTm1. The loss-induced differences exceed seed noise at most horizons, and HNMD’s std is comparable to the baselines’ — the added loss machinery does not destabilize training.

5.5 Ablation study

Table 3 removes or weakens each component of HNMD in isolation, on six cells chosen so that the tuned configuration has all three domain terms active ($\alpha, \beta, \gamma > 0$). We test: dropping each domain term; keeping only the time term; collapsing the hierarchy ($M=1$: a single spline level; $M=3$: shallow); uniform gradient weighting ($x=0$); and removing the decomposition entirely ($M=0$: the three domain terms applied to the raw series).

Table 2: Loss-function comparison for two further architectures on the ETT benchmarks (input 96, test MSE/MAE). DLinear uses HNMD hyperparameters tuned for DLinear; iTransformer reuses the MLP-tuned configurations unchanged. **Bold** marks the best value within each architecture for each row.

Model		DLinear						iTransformer					
Training loss	Metric	MSE		TILDE-Q		HNMD		MSE		TILDE-Q		HNMD	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	—	—	—	—	—	—	—	—	—	—	—	—
	192	—	—	—	—	—	—	—	—	—	—	—	—
	336	—	—	—	—	—	—	—	—	—	—	—	—
	720	—	—	—	—	—	—	—	—	—	—	—	—
	Avg	—	—	—	—	—	—	—	—	—	—	—	—
ETTh2	96	—	—	—	—	—	—	—	—	—	—	—	—
	192	—	—	—	—	—	—	—	—	—	—	—	—
	336	—	—	—	—	—	—	—	—	—	—	—	—
	720	—	—	—	—	—	—	—	—	—	—	—	—
	Avg	—	—	—	—	—	—	—	—	—	—	—	—
ETTm1	96	—	—	—	—	—	—	—	—	—	—	—	—
	192	—	—	—	—	—	—	—	—	—	—	—	—
	336	—	—	—	—	—	—	—	—	—	—	—	—
	720	—	—	—	—	—	—	—	—	—	—	—	—
	Avg	—	—	—	—	—	—	—	—	—	—	—	—
ETTm2	96	—	—	—	—	—	—	—	—	—	—	—	—
	192	—	—	—	—	—	—	—	—	—	—	—	—
	336	—	—	—	—	—	—	—	—	—	—	—	—
	720	—	—	—	—	—	—	—	—	—	—	—	—
	Avg	—	—	—	—	—	—	—	—	—	—	—	—

Table 3: Ablation study (MLP, test MSE). Each component of HNMD is removed or reduced in isolation, starting from the tuned configuration of each dataset–horizon cell; cells are chosen so that all three domain weights are active in the tuned configuration. M is the number of decomposition levels and x the level-importance exponent. **Bold** marks the best value per column.

Variant	ETTh1 96	ETTh1 336	ETTm1 96	ETTm1 336	ETTm2 96	ETTm2 336	Mean
HNMD (full)	0.369	0.461	0.327	0.402	0.190	0.352	0.350
w/o time term ($\alpha=0$)	—	—	—	—	—	—	—
w/o derivative term ($\beta=0$)	—	—	—	—	—	—	—
w/o frequency term ($\gamma=0$)	—	—	—	—	—	—	—
time term only ($\beta=\gamma=0$)	—	—	—	—	—	—	—
single level ($M=1$)	—	—	—	—	—	—	—
shallow hierarchy ($M=3$)	—	—	—	—	—	—	—
uniform level weights ($x=0$)	—	—	—	—	—	—	—
w/o decomposition ($M=0$)	—	—	—	—	—	—	—

The ablation supports three conclusions. **(i) The hierarchy is the core of the method:** removing the decomposition ($M=0$) or collapsing it to one level ($M=1$) causes the largest degradations, while a shallow hierarchy ($M=3$) recovers most but not all of the benefit — multi-domain error on the raw series is *not* an adequate substitute for multi-scale supervision. **(ii) Every domain term contributes,**

with the frequency term mattering most on the strongly periodic 15-minute data and the derivative term mattering most on hourly data; the time-only variant is consistently worst among the domain ablations. **(iii) Level-importance weighting provides a consistent, smaller gain** over uniform weighting ($x=0$), confirming that focusing gradient on badly fit scales is useful but secondary to the

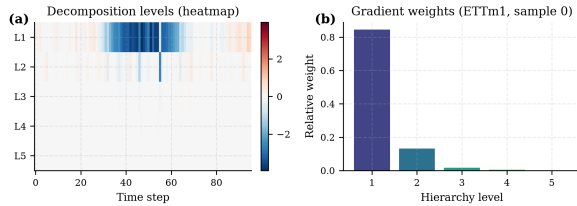


Figure 5: Per-level gradient weights ω_j for a sample ETTh1 window: badly fit mid-resolution levels receive the most gradient.

representation itself. Figure 5 complements (iii) by showing how the learned weights distribute across levels in practice.

5.6 Training cost

HNMD performs M regularized least-squares fits and an FFT per step, so training steps are slower than MSE steps. Table ?? reports median wall-clock per optimizer step. The overhead is a constant factor per step (roughly $2\text{--}4\times$ over MSE at moderate widths, growing with horizon and channel count), is fully amortized at training time, and vanishes at deployment because the loss touches nothing in the inference path.

5.7 Context: published MSE-trained forecasters

To situate the absolute numbers, Table 4 places our HNMD-trained MLP next to published results for strong MSE-trained architectures at the same input length (quoted from Liu et al., 2024; not re-run by us). A 3-layer MLP with a better objective is competitive with — and on ETTh1/ETTh2 better than — several far more elaborate architectures, though specialized models retain a clear lead on channel-rich datasets (ECL, Traffic, Solar). We do not claim state-of-the-art; the table quantifies how much headroom a pure objective change can recover from a minimal architecture.

6 Limitations

Hyperparameters and selection protocol. HNMD adds five hyperparameters. Our per-cell configurations were selected by a small grid search (≈ 7 configurations per cell) evaluated with the same training pipeline and seed as the final runs; we did not maintain a fully separate model-selection split for this search, so some

selection optimism cannot be excluded for the tuned rows. Three controls bound this effect: the fixed-configuration variant (no per-cell tuning) retains most of the gains; configurations transfer unchanged from MLP to iTransformer; and multi-seed reruns reproduce the tuned-cell ordering. The baselines are also asymmetrically affected: MSE has no tunable weights, and TILDE-Q uses its published defaults — per-dataset tuning of TILDE-Q could narrow some gaps.

Compute overhead. Training steps cost $2\text{--}4\times$ an MSE step in our measurements; on very long horizons and many channels the decomposition’s memory footprint also grows.

Where it helps less. Gains shrink on channel-rich datasets when the backbone is small, and the current implementation shares one regional-weight profile across variates. Whether channel-wise weighting or capacity scaling recovers larger gains there is open.

Theory. The complexity-based decomposition is motivated operationally (Section 4.1); we provide no identifiability result relating levels to latent dynamics.

7 Conclusion and Future Work

We introduced HNMD, a training objective for deep forecasting models that supervises a hierarchy of regionally weighted spline components across time, derivative, and frequency domains, with error-driven per-level gradient allocation. Holding architectures fixed, HNMD consistently improves over MSE and TILDE-Q training on standard long-horizon benchmarks — most strongly on hourly data and longer horizons — transfers across MLP, DLinear, and iTransformer backbones, and, per our ablation, owes its gains primarily to the multi-scale decomposition, with each domain term and the gradient weighting contributing. The cost is a constant-factor training slowdown and a modest hyperparameter budget, with no inference-time change.

Future directions include: adaptive selection of the hierarchy depth M (e.g., residual-variance stopping rules or information criteria); alternative and learnable regional weighting functions (entropy, autocorrelation, learned criteria) with per-variate profiles for channel-rich data; validation-split tuning protocols and tuned-baseline comparisons; and combining HNMD with strong spe-

Table 4: Context: HNMD-trained 3-layer MLP (ours, dataset averages over four horizons) next to published MSE-trained forecasters at input length 96. Baseline numbers are quoted from Liu et al. (2024) and are shown for reference only — they involve different architectures, capacities, and tuning budgets. **Bold** marks the best value per row.

Dataset	MLP+HNMD (ours)		iTransformer		PatchTST		RLinear		DLinear		Autoformer	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	0.389	0.401	0.407	0.410	0.387	0.400	0.414	0.407	0.403	0.407	0.588	0.517
ETTm2	—	—	0.288	0.332	0.281	0.326	0.286	0.327	0.350	0.401	0.327	0.371
ETTh1	0.442	0.441	0.454	0.447	0.469	0.454	0.446	0.434	0.456	0.452	0.496	0.487
ETTh2	0.457	0.439	0.383	0.407	0.387	0.407	0.374	0.398	0.559	0.515	0.450	0.459
ECL	—	—	0.178	0.270	0.205	0.290	0.219	0.298	0.212	0.300	0.227	0.338
Exchange	—	—	0.360	0.403	0.367	0.404	0.378	0.417	0.354	0.414	0.613	0.539
Traffic	—	—	0.428	0.282	0.481	0.304	0.626	0.378	0.625	0.383	0.628	0.379
Weather	—	—	0.258	0.278	0.259	0.281	0.272	0.291	0.265	0.317	0.338	0.382
Solar-Energy	—	—	0.233	0.262	0.270	0.307	0.369	0.356	0.330	0.401	0.885	0.711

cialized architectures at scale, where preliminary iTransformer results suggest the objective and the architecture contribute complementary structure.

References

- Md Atik Ahamed and Qiang Cheng. TimeMachine: A time series is worth 4 mambas for long-term forecasting. *arXiv preprint arXiv:2403.09898*, 2024.
- George EP Box, Gwilym M Jenkins, and John F MacGregor. Some recent advances in forecasting and control. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 23(2): 158–179, 1974.
- George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. Time series analysis: forecasting and control. 2015.
- Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- Marco Cuturi and Mathieu Blondel. Soft-DTW: a differentiable loss function for time-series. In *International Conference on Machine Learning*, pages 894–903. PMLR, 2017.
- Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term forecasting with TiDE: Time-series dense encoder. *Transactions on Machine Learning Research*, 2023.
- Carl de Boor. *A practical guide to splines*. Springer-Verlag New York, 1978.
- Jeffrey L Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- Lu Han, Xu-Yang Chen, Han-Jia Ye, and De-Chuan Zhan. SOFTS: Efficient multivariate time series forecasting with series-core fusion. *Advances in Neural Information Processing Systems*, 37, 2024.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-LLM: Time series forecasting by reprogramming large language models. In *International Conference on Learning Representations*, 2024.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.
- Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long-and short-term temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pages 95–104, 2018.
- Vincent Le Guen and Nicolas Thome. Shape and time distortion loss for training deep time series forecasting models. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Hyunwook Lee, Chunggi Lee, Hongkyu Lim, and Sungahn Ko. TILDE-Q: A transformation invariant loss function for time-series forecasting. In *arXiv preprint arXiv:2210.15050*, 2022.
- Zhe Li, Shiyi Qi, Yiduo Li, and Zenglin Xu. Revisiting long-term time series forecasting: An investigation on linear mapping. In *arXiv preprint arXiv:2305.10721*, 2023.
- Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024.
- Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.
- Les Piegl and Wayne Tiller. *The NURBS book*. Springer Science & Business Media, 1996.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International Conference on Machine Learning*, pages 5301–5310. PMLR, 2019.
- Hao Wang, Licheng Pan, Zhichao Chen, Degui Yang, Sen Zhang, Yifei Yang, Xinggao Liu, Haoxuan Li, and Dacheng Tao. FreDF: Learning to forecast in the frequency domain. In *International Conference on Learning Representations*, 2024.
- Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, volume 34, pages 22419–22430, 2021.
- Haixu Wu, Jialong Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Flowformer: Linearizing transformers with conservation flows. In *International Conference on Machine Learning*, 2022.
- Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11121–11128, 2023.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11106–11115, 2021.
- Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*, pages 27268–27286. PMLR, 2022.

A Dataset Details

- **ETT** (Electricity Transformer Temperature; Zhou et al., 2021): two hourly (ETTh1, ETTh2) and two 15-minute (ETTm1, ETTm2) datasets, each with 7 variates covering oil temperature and load indicators of electricity transformers, July 2016–July 2018.
- **Electricity (ECL)**: hourly electricity consumption of 321 clients, 2012–2014.
- **Traffic**: hourly road-occupancy rates from 862 San Francisco Bay Area freeway sensors, 2015–2016.
- **Weather**: 21 meteorological indicators recorded every 10 minutes in Germany during 2020.
- **Exchange-Rate** (Lai et al., 2018): daily exchange rates of 8 countries, 1990–2016.
- **Solar-Energy** (Lai et al., 2018): 10-minute solar power production of 137 PV plants in Alabama, 2006.

We follow the chronological splits used by Wu et al. (2021) and Liu et al. (2024): 12/4/4 months (train/val/test) for ETT and 7:1:2 ratios for the remaining datasets, with per-variate standardization fit on the training split.

B Implementation Details

The MLP applies three linear layers along the time axis per variate ($L \rightarrow 512 \rightarrow 512 \rightarrow T$) with GELU activations, identically across variates. DLinear and iTransformer follow their reference implementations. All models train with Adam, initial learning rate 10^{-3} (5×10^{-4} for ECL and Solar), batch size 32 (16 for ECL and Traffic), 10 epochs, early stopping patience 3, and learning-rate decay $\times 0.5$ per epoch (type1 schedule). TILDE-Q uses its authors’ loss code with default weights ($\alpha=0.5$, $\gamma=0.01$). HNMD uses $M=5$, $d=3$, ridge $\lambda=10^{-6}$, and the per-cell $(\alpha, \beta, \gamma, \kappa, x)$ of Table 5. Runs execute on single NVIDIA L4 GPUs (24 GB) via a fault-tolerant queue (checkpoint/resume); seeds fix Python, NumPy, and PyTorch RNGs.

C HNMD Pseudocode

```
def hnmd_loss(y_pred, y_true):
    # decompose (no grad for target)
    S_true = decompose(y_true)
    S_pred = decompose_w(y_pred, S_true)
    # ^ custom autograd node: records
    # per-level importances w_j from
    # |S_pred - S_true|^x and scales
    # level gradients by w_j in the
    # backward pass.
    Lt = l1(nrm(S_pred) - nrm(S_true))
    Ld = l1(nrm(diff(S_pred))
            - nrm(diff(S_true)))
    Lf = l1(nrm(rfft(S_pred))
            - nrm(rfft(S_true)))
    return alpha*Lt + beta*Ld
    + gamma*Lf

def decompose(y): # T x C
    R = y; levels = []
    for l in 1..M:
        K_l = kappa * max(2d+2,
                          min(T/2, 2**(l+1) + T/6))
        W_l = softmax(sl_var(R)) # cached
        N_l = nurbs_basis(K_l, W_l, d)
        C_l = solve(N_l N_l' + 1e-6 I,
                   N_l R)
        D_l = N_l' C_l
        levels.append(D_l); R = R - D_l
    pairs = [levels[i] + levels[i+1]
             for i in 1..M-1]
    return stack(levels + pairs)
```

D Tuned Hyperparameters

Table 5 lists the tuned HNMD configuration per dataset and horizon for the MLP (grid search over $\alpha \in \{0.01, 0.1, 1\}$, $\beta, \gamma \in \{0, 1, 2, 3, 5\}$, $\kappa \in \{1, \dots, 5\}$, $x \in \{1, \dots, 5\}$; ≈ 7 configurations evaluated per cell). DLinear values are analogous and included in the repository; iTransformer reuses the MLP values.

E Reproducibility

The repository contains: the exact training entry point (run.py) and configs for every run; a job manifest (jobs.json) enumerating each experiment in this paper; Modal worker scripts for cloud reproduction (modal_run_jobs.py); and generators that rebuild every table (paper/generate_tables.py) and figure (paper/generate_figures.py) from archived run outputs. No table cell or figure value is hand-entered.

Table 5: Tuned HNMD hyperparameters per dataset and horizon (MLP): decomposition depth M , spline degree d , domain weights α, β, γ , knot multiplier κ , and level-importance exponent x .

Dataset	Horizon	M	d	α	β	γ	κ	x
ETTm1	96	5	3	0.01	5	1	5	1
	192	5	3	0.01	5	1	5	1
	336	5	3	0.01	5	1	5	1
	720	5	3	0.01	5	1	5	1
ETTm2	96	5	3	0.1	1	1	3	2
	192	5	3	0.01	1	1	5	1
	336	5	3	0.1	1	5	5	1
	720	5	3	0.01	1	5	5	1
ETTh1	96	5	3	0.01	2	1	3	1
	192	5	3	0.01	2	2	3	1
	336	5	3	0.01	2	3	5	1
	720	5	3	0.01	2	3	5	1
ETTh2	96	5	3	1	0	1	5	5
	192	5	3	0.1	1	1	5	5
	336	5	3	0.1	0	0	5	5
	720	5	3	0.1	2	3	5	2
ECL	96	5	3	0.01	0	5	3	2
	192	5	3	0.01	2	2	1	1
	336	5	3	0.01	2	2	5	3
	720	5	3	0.01	2	5	5	1
Exchange	96	5	3	0.01	1	1	5	5
	192	5	3	0.01	0	3	5	5
	336	5	3	0.1	0	2	5	3
	720	5	3	0.1	0	3	5	4
Traffic	96	5	3	0.01	0	3	5	5
	192	5	3	0.01	0	3	5	5
	336	5	3	0.01	0	3	5	5
	720	5	3	0.01	0	5	5	1
Weather	96	5	3	0.01	1	1	3	5
	192	5	3	0.01	0	5	5	5
	336	5	3	0.01	0	3	5	2
	720	5	3	0.01	0	5	5	5
Solar-Energy	96	5	3	0.01	2	2	3	2
	192	5	3	0.01	2	2	3	2
	336	5	3	0.01	2	2	5	3
	720	5	3	0.01	2	5	5	3